

POLITEKNIK BESUT

MYHEP STUDENT AFFAIRS MANAGEMENT SYSTEM

TECHNICAL REPORT

OFFICIAL PROJECT TITLE: [CONFIRM TITLE]

PREPARED BY: [STUDENT NAME(S) AND REGISTRATION ID(S)]

SUPERVISOR: [SUPERVISOR NAME]

PROGRAMME: [PROGRAMME / DEPARTMENT]

ACADEMIC SESSION: [SESSION]

SUBMISSION DATE: [DATE]

DRAFT FOR AUTHOR COMPLETION

This document is an editable technical-report draft based on the MyHEP/StudentEdge repository and the supplied report guidelines. Replace every bracketed field and obtain institutional approval before submission.

DECLARATION

Complete this declaration only after the listed author or authors have reviewed the report and confirmed its originality and accuracy. Do not sign while any placeholder or unverified statement remains.

I/We, [AUTHOR NAME(S)], declare that this technical report describes the MyHEP Student Affairs Management System work carried out by [AUTHOR / PROJECT TEAM] under the supervision of [SUPERVISOR]. The report is my/our own work except where sources are cited. I/We have identified repository-derived evidence and have not represented unperformed user-acceptance testing as completed.

Name and registration ID: _____

Signature: _____ Date: _____

Supervisor name and signature: _____

APPROVAL

This report is submitted in partial fulfilment of the requirements for [PROGRAMME / COURSE]. Approval details must be completed by the institution.

Approval record

Role	Name	Signature	Date
Student author(s)	[NAME(S)]	_____	_____
Project supervisor	[NAME]	_____	_____
Programme coordinator	[NAME]	_____	_____

ABSTRAK

MyHEP ialah sistem pengurusan hal ehwal pelajar berasaskan web untuk menyokong urusan biasiswa dan kebajikan, disiplin, pergerakan pelajar, aktiviti program serta perkhidmatan sendiri. Laporan ini menghuraikan keperluan, reka bentuk, pelaksanaan berasaskan Laravel, kawalan akses mengikut peranan dan pelan pengujian berdasarkan artifak repositori. Pengesahan automatik semasa menunjukkan ujian PHP dan binaan antaramuka berjaya. Keputusan UAT sebenar, metodologi projek, jadual dan butiran rasmi pengarang masih perlu dilengkapi dan disahkan sebelum penyerahan.

Kata kunci: MyHEP, hal ehwal pelajar, Laravel, pengujian perisian

ABSTRACT

MyHEP is a web-based student-affairs management system supporting scholarship and welfare administration, discipline, student movement, programme activities and student self-service. This report describes its requirements, design, Laravel implementation, role-scoped access controls and test plan using repository evidence. Current automated verification records a passing PHP test suite and frontend production build. Actual user-acceptance results, the project methodology, schedule and official author details remain to be completed and approved before submission.

Keywords: MyHEP, student affairs, Laravel, software testing

TABLE OF CONTENTS

Declaration.....	2
Approval	3
Abstrak and Abstract	4
List of Tables	6
List of Figures.....	6
1.0 Project Plan	7
2.0 Requirements Specification	13
3.0 Final Design	19
4.0 Test Description and Results.....	25
5.0 Major Findings and Discussion.....	30
6.0 Recommendation and Conclusion	35
References.....	37
Appendices.....	38

Page references reflect this draft and must be updated after substantive editing.

LIST OF TABLES

Table 1.1	Related work comparison
Table 1.2	Draft project schedule
Table 2.1	User roles and responsibilities
Table 2.2	Functional requirements
Table 2.3	Non-functional requirements
Table 2.4	Proposed deployment configuration
Table 2.5	Business rule examples
Table 3.1	Main system components
Table 3.2	Conceptual data groups
Table 3.3	Access-scope summary
Table 4.1	Current validation environment
Table 4.2	Automated validation results
Table 4.3	Planned functional test cases
Table 4.4	User-acceptance test plan
Table 5.1	Risks and controls
Table 5.2	Candidate measures for a controlled pilot
Table A.1	Requirements traceability
Table B.1	Author completion checklist

LIST OF FIGURES

Figure 3.1	MyHEP high-level architecture
Figure 3.2	Core student-affairs workflow
Figure 3.3	Programme participation workflow

1.0 PROJECT PLAN

1.1 Project Background

Student-affairs units handle information that spans student identity, financial assistance, welfare, conduct, campus movement and participation in institutional programmes. When these activities sit in separate forms, spreadsheets or inboxes, staff may repeat data entry, lose the link between a decision and its evidence, or struggle to give students a clear view of their own records. These are common process risks, not measured outcomes for MyHEP; local baseline measurements have not been supplied.

The repository describes MyHEP, also labelled StudentEdge in project documentation, as a Laravel platform for Politeknik Besut. It joins staff administration with student self-service. Staff access is differentiated by role and scope. The student portal exposes permitted personal records, programme information, documents, profile settings and selected applications. Administrators manage records and workflows within their assigned responsibilities, while system administrators configure the platform.

This report treats repository documentation and code as evidence of the implemented design. It does not treat the visual screenshots or product description as proof that every live workflow is correct. Automated tests and a production frontend build are recorded separately from production database verification, third-party delivery checks and user-acceptance testing.

1.2 Problem Statement

The project addresses the need for a single, role-aware system for routine student-affairs work. The identified operational concerns are fragmented records, manual hand-offs, limited student visibility and the difficulty of maintaining traceable decisions across scholarship, welfare, discipline, movement and event workflows. The project documentation establishes the intended functions but does not provide a measured pre-implementation study, a formal sample, or verified time-and-error baselines. Consequently, this report frames the problem as a system requirement and not as a quantified institutional finding.

The engineering problem is also one of safe consolidation. The system handles sensitive student, household, financial and conduct information. A shared platform must prevent users from viewing or changing records outside their role, preserve evidence for

later review, and keep critical workflows understandable on desktop and mobile devices. The design therefore needs access checks at server-side routes and data operations, not only hidden menu items.

1.3 Project Aim and Objectives

The aim is to document and evaluate the current MyHEP implementation as a role-scoped student-affairs management platform for Politeknik Besut.

1. Analyse the documented user roles, business workflows and information requirements represented in the MyHEP repository.
2. Describe the implemented architecture, data groups, security boundaries and user-facing design using source-aligned evidence.
3. Verify the available automated and build checks, and prepare a structured acceptance plan for evaluation by actual users.

1.4 Project Scope

The software scope covers the modules represented in the system overview and route, model, migration and test trees. It includes authentication and account support; student records and protected documents; scholarship and welfare; discipline and fines; campus movement and QR checkpoints; laptop inventory and loans; programme registration, attendance, surveys, points, reports and certificates; notifications; dashboards; operational administration; and AI-assisted support or reporting where configured.

The role scope includes students, lecturers and staff, scholarship administrators, discipline administrators, the Head of Student Affairs, system administrators and guards. Permissions are not interchangeable. For example, scholarship personnel work with welfare and financial-assistance information, discipline personnel work with conduct-related records, and guards use checkpoint functions. Exact access remains governed by routes, middleware, policy checks and record ownership.

The technical scope is the current web application. The repository identifies Laravel 13, PHP 8.3 or later, MySQL/MariaDB for normal operation, Blade views with JavaScript and CSS, and Vite/Tailwind for frontend assets. The current test run uses SQLite in memory; it does not validate the production MySQL schema. A future Flutter client is outside the implemented scope of this report and should use a documented API rather than directly connect to the database.

1.5 Project Constraints and Assumptions

- The official project title, author names, student IDs, supervisor, programme and submission session are not present in the reviewed repository materials; placeholders remain on the preliminary pages.
- The formal development methodology and project schedule were not confirmed. The report includes an empty schedule template and does not claim that a particular methodology was used.
- The production database state, deployed migration status, mail and push delivery, AI provider availability and backup restoration were not tested in this report.
- The automated suite is evidence of the configured test environment only. It is not a substitute for operational acceptance by students and staff.
- Repository documentation has its own baseline date. This report prioritizes the current code and current automated checks where they are available.

1.6 Literature Review

Student-affairs platforms combine administrative records with services used directly by students. Tam et al. describe requirements analysis for a student-affairs management information system using Agile requirements engineering. Their work provides a relevant example of deriving requirements from users and organizing them as a product backlog; it does not establish that MyHEP followed Scrum or the same research process (Tam et al., 2022).

Attendance systems illustrate one workflow where a digital service can reduce manual recording while introducing new validation and privacy questions. Aldabagh's review compares attendance approaches, including biometric and non-biometric systems such as QR-based methods. It supports treating attendance as a distinct workflow with usability, reliability and deployment trade-offs rather than assuming that digitization alone guarantees accuracy (Aldabagh, 2024).

Abdellatif et al. (2022) examined a location-linked QR attendance approach with medical students and reported participants' perceptions of its usability and related qualities. This study is relevant to MyHEP's QR attendance and movement features, but its participants, institutional setting and implementation differ. Its findings should not be generalized to MyHEP users without local testing.

Together, these sources suggest three design considerations: elicit role-specific requirements, validate attendance workflows with the intended users and environment, and

evaluate privacy and usability alongside functional correctness. The MyHEP repository includes role-scoped workflows and automated tests. It does not contain a completed local user study that would establish user satisfaction or an impact on processing time.

Table 1.1 Related work comparison

Source	Focus	Relevance to MyHEP	Transfer limit
Tam et al. (2022)	Student-affairs requirements using Agile requirements engineering	Supports explicit stakeholder requirements and backlog traceability	Does not prove MyHEP used Agile or validate its requirements
Aldabagh (2024)	Review of student attendance-management approaches	Informs attendance choices and their operational trade-offs	A review is not an evaluation of MyHEP
Abdellatif et al. (2022)	Location-linked QR attendance and student perceptions	Relevant to QR attendance and location validation	Different sample and context; local UAT is still needed

1.7 Development Methodology

The repository does not identify the methodology actually used for the project. The author must insert the approved method and provide evidence such as project meeting records, requirements versions, sprint records, prototypes or review notes. This draft therefore does not label development as Agile, Waterfall or prototyping. The final report should explain how requirements were gathered, how the system was designed and implemented, how changes were reviewed, and how testing informed revisions.

If the project team confirms an iterative method, describe the actual iteration length, activities, participants and delivered increments. If a sequential method was used, describe its actual phases and approval points. Do not retrofit a method name merely because the code or report can be organized into phases.

1.8 Draft Project Schedule

The following is a completion template only. Replace phase dates, durations, dependencies and completion status with the approved project plan. No dates or completion percentages are inferred from repository timestamps.

Table 1.2 Draft project schedule

Activity	Start	End	Evidence / deliverable	Status
Problem definition and approval	[DATE]	[DATE]	Approved problem statement	[TBC]
Requirements elicitation	[DATE]	[DATE]	Interview / observation /	[TBC]

Activity	Start	End	Evidence / deliverable	Status
			document evidence	
Design and review	[DATE]	[DATE]	Approved designs and data model	[TBC]
Implementation iterations	[DATE]	[DATE]	Versioned software increments	[TBC]
Verification and defect correction	[DATE]	[DATE]	Test records and resolved issues	[TBC]
User acceptance and evaluation	[DATE]	[DATE]	Signed UAT evidence	[TBC]
Report and final approval	[DATE]	[DATE]	Approved report and demonstration	[TBC]

1.9 Stakeholder Needs and Elicitation

A final needs analysis should include students and each staff group that performs or approves an in-scope workflow. Students can explain where self-service is unclear, which notifications are useful and what information they expect to see. Scholarship and welfare staff can describe eligibility checks, evidence handling and review outcomes. Discipline staff can clarify case status, evidence access and appeal or payment paths. Guards can describe checkpoint conditions, device availability and safe fallback procedures. System administrators can identify account lifecycle, permission review, deployment and recovery requirements.

The reviewed repository supplies a role model and documented workflows, but it does not include interview transcripts, survey results or signed requirements approvals. The team should record the date and method of each consultation, participant role, questions asked, themes found, resulting requirement and approval. Conflicting needs should be resolved by the institution's policy owner, particularly when convenience conflicts with privacy or separation of duties.

Requirements should be written so a tester can observe success or failure. "The system is user-friendly" is not directly testable. A stronger acceptance statement names the task, user role, completion condition and evidence. For example, a student can view only their own permitted document list; another student attempting the same request receives no document content. The final threshold and supported devices must be decided locally.

1.10 Feasibility and Project Boundaries

Technical feasibility is supported by the existing Laravel application, database migrations, browser interface and automated test suite. The fresh frontend build and PHP test run show that the checked source can compile assets and execute automated tests in the local environment. They do not establish the capacity or availability of a production host, the correctness of a live database migration, or the readiness of external services. Those require deployment-specific evidence.

Operational feasibility depends on staff ownership of account provisioning, permission review, workflow policies, templates and support. Training and a documented manual fallback will be necessary when cameras, networks, queue workers or external services are unavailable. The repository alone cannot establish whether departments have approved operating procedures or allocated support time.

Economic feasibility should compare implementation and hosting costs with the institution's existing process costs. No project budget, support-cost estimate, volume baseline or quantified benefit analysis was supplied; the report therefore makes no return-on-investment claim. The team can collect a baseline for staff processing time, correction rates, duplicate entry and support contacts, then repeat the same measurements after a controlled pilot.

Legal and policy feasibility requires institutional review of personal-data handling, retention, access logging, consent notices, provider terms and records disclosure. This report identifies the relevant design questions but does not give a legal opinion. Scope should remain limited to approved workflows until those responsibilities and data-sharing rules are clear.

2.0 REQUIREMENTS SPECIFICATION

2.1 Stakeholders and User Roles

MyHEP serves users with different operational responsibilities. A role name alone is not sufficient to authorize access: the application also needs the appropriate module scope, record ownership and request context. The matrix below summarizes documented intent. The final project team should confirm it with the institution's approved access policy.

Table 2.1 User roles and responsibilities

Role	Primary responsibilities	Data boundary
Student	View permitted profile and personal records; participate in programmes; submit allowed forms and applications	Own account and records explicitly made available to the student
Lecturer / staff	Use permitted programme, reporting and operational pages	Assigned modules, programmes or enabled actions
Scholarship administrator	Manage scholarship, B40, welfare and related student information	Scholarship and welfare scope
Discipline administrator	Manage offenses, fines, evidence, stickers and related workflows	Discipline scope
Head of Student Affairs	Oversee student-affairs activities and account workflows	Broad approved HEP scope
System administrator	Configure accounts, features, platform settings and operations	System control; sensitive access should be audited
Guard	Use movement checkpoints and permitted lookup operations	Checkpoint and guard-specific scope

2.2 Functional Requirements

Requirements below summarize current documented capabilities. “Implemented” means the repository describes or contains the feature; it does not mean every branch has passed independent acceptance testing. The traceability appendix links these requirements to representative evidence or validation still required.

Table 2.2 Functional requirements

ID	Requirement	Module
FR-01	The system shall authenticate users and provide account recovery functions.	Authentication / account
FR-02	The system shall enforce server-side role and module-scope checks for protected routes and actions.	Authorization
FR-03	The system shall let users manage supported locale, theme and	Shared interface

ID	Requirement	Module
	account preferences.	
FR-04	The system shall support student profile and profile-completion workflows.	Student records
FR-05	The system shall store and present scholarship and welfare records to authorized users.	Scholarship / welfare
FR-06	The system shall support scholarship-status forms and related announcements.	Scholarship
FR-07	The system shall manage discipline records, evidence, fines and student applications.	Discipline
FR-08	The system shall manage vehicle stickers and related status where configured.	Discipline
FR-09	The system shall record campus movement using permitted checkpoint workflows.	Movement
FR-10	The system shall support QR-based checkpoint or attendance actions where configured.	QR workflows
FR-11	The system shall manage programme records and participant registration.	Programmes
FR-12	The system shall support programme attendance modes and questionnaire responses.	Programmes
FR-13	The system shall support optional geofence checks when enabled.	Programmes
FR-14	The system shall calculate or record eligible internal-student participation points.	Points
FR-15	The system shall generate programme reports from authorized programme data.	Reporting
FR-16	The system shall support certificate records and queued bulk generation where configured.	Certificates
FR-17	The system shall manage laptop inventory, borrowing and returns.	Laptop loans
FR-18	The system shall provide authorized access to private student documents.	Documents
FR-19	The system shall provide dashboards and operational monitoring pages.	Administration
FR-20	The system shall record supported notifications and allow users to review active sessions.	Account / operations
FR-21	The system shall allow support-problem reporting through the available workflow.	Support
FR-22	The system shall expose AI-helper or AI-assisted report functions only when provider configuration is available.	AI integration
FR-23	The system shall allow system administrators to manage supported settings, feature controls and maintenance operations.	System administration
FR-24	The system shall provide English and Bahasa Melayu interface catalogues for supported screens.	Localization

2.3 Non-Functional Requirements

The following quality requirements are design and acceptance targets. The repository does not provide measured service-level objectives for response time, uptime, accessibility or recovery. The project owner must approve quantitative thresholds before performance or operational compliance can be claimed.

Table 2.3 Non-functional requirements

ID	Quality	Requirement / verification
NFR-01	Security	Protected routes and data actions must enforce authentication, role scope and record ownership server-side. Verify unauthorized requests are denied.
NFR-02	Privacy	Collect only information needed for a workflow; restrict household, financial, conduct and identity data to authorized roles.
NFR-03	Auditability	Important administrative decisions should preserve the actor, affected record, time and decision evidence where the feature requires it.
NFR-04	Usability	Students and staff should complete their role-specific tasks on supported desktop and mobile layouts. Verify with representative users.
NFR-05	Accessibility	Controls should have labels, keyboard access, readable contrast and understandable validation feedback. Run a documented accessibility review.
NFR-06	Performance	Define response-time and concurrent-user targets with the institution; test against a production-like dataset and server.
NFR-07	Availability	Define service hours, maintenance windows and recovery objectives; monitor the deployed service.
NFR-08	Compatibility	Support the institution-approved browser and device matrix; validate the installed PWA separately from ordinary browser use.
NFR-09	Maintainability	Keep business rules testable and gradually move large route-closure workflows into focused controllers or services.
NFR-10	Reliability	Handle mail, push, AI and queued work failures without losing the primary record or misreporting completion.
NFR-11	Localization	Provide correct English and Bahasa Melayu text for supported user-facing flows.
NFR-12	Backup and recovery	Document database and private-file backup, restore and verification procedures, then execute a restore exercise.

2.4 Software and Hardware Requirements

The source repository declares PHP 8.3 or later and Laravel 13, with MySQL or MariaDB for normal application use. The frontend uses Vite 8 and Tailwind CSS 4. The

validation run for this report used PHP 8.4.0, Laravel 13.24.0, SQLite in-memory for tests, and a successful Vite 8.2.1 production build. These are observed development/test versions, not a complete production server specification.

A server deployment needs a supported PHP runtime, required PHP extensions, a web server configured for Laravel's public directory, a production database, secure environment configuration, writable cache and storage locations, and a process manager for any queue-backed tasks. Email, push and AI features require separately configured external services. Exact CPU, memory, storage, traffic and backup capacity must be selected from institution load estimates and validated under test.

Users need a current supported browser on desktop or mobile. The repository contains PWA and QR-related functionality; device camera permission, secure-context rules, installed-PWA behavior and native Flutter support require separate device testing. Flutter is not the client framework of this web implementation.

2.5 System Configuration

Table 2.4 Proposed deployment configuration

Component	Current evidence	Production confirmation required
Application	Laravel 13 web application; server-rendered Blade plus JavaScript/CSS	Web server, PHP extensions, environment, error handling and release procedure
Database	MySQL/MariaDB declared for normal use; PHPUnit uses SQLite memory	Engine/version, schema migration plan, least-privilege account, backup and restore
Frontend assets	Vite/Tailwind; fresh production build passed	Manifest deployment, cache policy and rollback
Private file storage	Authorized controller routes for private downloads described	Storage permissions, retention, malware checks and backup
Queue processing	Queue-backed work used for bulk certificate generation and integrations	Worker supervision, retry/dead-letter policy and monitoring
External services	Mail, push and AI are server-side dependencies	Credentials, quotas, privacy terms, timeouts and delivery monitoring

2.6 Security and Privacy Requirements

Authorization must be checked on every protected request and on operations that access a particular record. Hiding a menu item is not authorization. Tests should cover role boundaries, object ownership, direct URL access, mass assignment, invalid state transitions

and requests made after a session is revoked. Sensitive exports and file downloads need the same scope enforcement as ordinary page views.

The application's information includes personally identifiable data and, in some workflows, household, scholarship, welfare, discipline or movement records. Access should be limited to staff with an institutional purpose. External AI prompts should exclude unnecessary sensitive information, consistent with the project's operational notes. Logs should avoid storing secrets or full sensitive payloads. Retention, disclosure and lawful processing policy must be confirmed with the institution; this report does not make a legal-compliance determination.

Operational controls include protected secret storage, HTTPS, least-privilege database credentials, timely dependency updates, review of Composer security advisories, secure session configuration, monitored queue workers, and tested backup restoration. A passing code suite does not confirm that production controls are configured correctly.

2.7 Business Rules and Validation

Each workflow should define its valid states, actor, preconditions, allowed transition and resulting record. The repository contains status-oriented flows for review and operations, but the institution should maintain a concise policy table for each sensitive process. The author should verify these rules with policy owners before treating the requirements as final. Validation belongs on the server even when the browser also gives immediate feedback. This protects the system from crafted requests, stale forms and clients that do not execute interface JavaScript.

Table 2.5 Business rule examples for institutional confirmation

Area	Rule to confirm	Evidence / control
Profile completion	Which fields are required and which approved exceptions permit access before completion?	Policy owner, validation tests and exception audit
Scholarship	Which eligibility inputs and supporting documents are required for each status?	Approved criteria, reviewer record and student notice
Discipline	Which role may create, review, amend or close a case?	State-transition matrix and role-boundary tests
Attendance	How are late scans, duplicate scans, proxy scans and offline conditions handled?	Approved event rule and test scenarios
Points	Which participants qualify and how are corrections or reversals authorized?	Point policy, recalculation test and audit record
Documents	Who may upload, download, replace or remove an item and for how long?	Retention policy and access-denial tests

Area	Rule to confirm	Evidence / control
Certificates	Which template, signatory, identity fields and serial rules are official?	Approved artwork, issuance register and revocation process

Errors should be actionable and should not reveal sensitive information. A validation response can identify a missing field without exposing another student's record or internal security logic. When an external service fails, the application should distinguish "queued," "sent," "failed" and "delivery not confirmed." A successful request to a provider is not equivalent to a delivered message.

2.8 Inputs, Outputs and Records

Inputs include account credentials, profile changes, scholarship or welfare forms, supporting documents, attendance scans, movement requests, programme participation, staff decisions and administrative configuration. Input validation should check type, length, allowed values, state and authorization. Uploaded files should be restricted by content type and size, stored outside publicly served paths where appropriate, and returned only through an authorized download action.

Outputs include student-facing status, staff review queues, notifications, operational dashboards, exports, programme reports and certificates. Each output needs an intended audience and an owner who can approve its contents. Export filters and identifiers should be constrained to the caller's authorized scope. Generated documents should retain enough provenance to identify the source records and generation time while avoiding unnecessary sensitive data in file names or logs.

Retention and deletion behavior should follow an institution-approved records schedule. The application's ability to store a record does not mean the institution should retain it indefinitely. The data owner should identify which records must be preserved for audit or appeal, which can be archived, and who approves correction or deletion requests.

3.0 FINAL DESIGN

3.1 Design Overview

MyHEP uses a browser-based, server-rendered application. Laravel receives web requests, applies authentication and scoped access checks, invokes controller or route-level business logic, and reads or writes application data through Eloquent or the query builder. Blade renders pages, while JavaScript and compiled CSS add interactive behavior. File access, mail, push and AI operations pass through server-side code rather than exposing database credentials or provider secrets to the browser.

Figure 3.1 MyHEP high-level architecture

Layer	Responsibility	Representative technology
User interface	Student and staff pages, forms, dashboards and PWA shell	Browser, Blade, JavaScript, CSS
Application	Authentication, validation, policies, workflows and reports	Laravel routes, middleware, controllers, services
Data	Student-affairs records, relationships and migrations	Eloquent / query builder; MySQL or MariaDB
Background work	Queued generation and asynchronous operations	Laravel queues and supervised workers
External services	Email, push and AI provider integration	Server-side provider adapters / configuration

Figure 3.1 summarizes logical boundaries, not a deployment topology. The production hosting provider, network diagram, server counts and disaster-recovery topology were not confirmed in the reviewed materials.

3.2 System Context and Main Components

Students use the portal to review permitted information and submit supported actions. Staff members use only the administration modules granted to their role. Guards use movement checkpoints. System administrators configure accounts, platform behavior and operations. External services may deliver email, push notifications or AI responses, but the core application remains responsible for deciding what data may leave the system.

Table 3.1 Main system components

Component	Inputs	Outputs / responsibilities
Authentication and accounts	Credentials, recovery requests, session actions	Authenticated sessions, locale/theme choices, notifications

Component	Inputs	Outputs / responsibilities
Student records	Profile edits, imports, authorized staff updates	Student profile and permitted status data
Scholarship and welfare	Applications, forms, B40 inputs, staff decisions	Status, welfare records and announcements
Discipline	Offense data, evidence, applications, payments	Case status, fines, stickers and permitted exports
Movement	Student requests and checkpoint scans	Movement history, active/outside status and guard views
Programme operations	Programme details, registration, attendance and survey	Participant lists, attendance, points and report inputs
Document centre	Uploaded document and access request	Private document storage and authorized delivery
Operations and reporting	Authorized query parameters and system events	Dashboards, monitoring and generated report files

3.3 Access-Control Design

The access design separates authentication (who signed in) from authorization (what that person may do). Session middleware establishes authentication. Role and module scopes constrain route access. Record ownership and contextual checks further limit access to individual students, programmes, documents or cases. This layered pattern is especially important where a user can guess or manually edit a URL.

Table 3.3 Access-scope summary

Role	Example allowed scope	Review question
Student	Own profile and self-service workflows explicitly exposed	Can another student's identifier be substituted?
Lecturer / staff	Assigned programme or enabled staff operations	Are assignment and action-level checks enforced?
Scholarship administrator	Scholarship and welfare workflows	Are unrelated discipline records excluded?
Discipline administrator	Conduct, fine, evidence and sticker workflows	Are private attachments scoped as well?
Head of Student Affairs	Approved cross-module oversight and accounts	Are privileged changes auditable?
System administrator	Platform settings, feature and operations control	Are high-impact actions logged and protected?
Guard	Checkpoint-specific movement workflows and allowed lookup	Can the role reach administrative routes directly?

3.4 Student-Affairs Workflow

The normal lifecycle begins when an authorized account is created or imported. A student signs in and completes required profile information. The student may then use eligible scholarship, welfare, discipline, document or programme features. Staff review submissions through their scoped workspace and record a decision. The student sees only the resulting information that policy permits. Sensitive evidence remains available only through a protected access path.

Each workflow should preserve a clear relationship between the request, the affected student or programme, the reviewing user, the decision and any supporting document. Validation should reject incomplete or invalid transitions before persistence. Audit records should make subsequent review possible without copying sensitive contents into general logs.

Figure 3.2 Core student-affairs workflow

Step	Student action	Staff / system action
1. Account	Sign in and complete required profile information	Provision account and apply server-side role scope
2. Request	Submit an eligible form or join a programme	Validate fields, ownership and workflow state
3. Review	Receive permitted status and requests for correction	Assigned staff review evidence and record decision
4. Follow-up	Update the submission or view final status	Notify the student and preserve decision history

3.5 Programme Participation Workflow

A programme owner prepares the programme details and any required paperwork. The programme is reviewed or approved according to the configured process. Students register or join through the available channel. Attendance may use a selected mode, including QR where configured. A questionnaire may collect programme feedback. Eligible internal students can receive participation points. Authorized staff then generate a report, review its content and manage certificates for eligible records.

Figure 3.3 Programme participation workflow

Stage	System action	Control / evidence
Prepare	Create programme and supporting details	Owner and required fields

Stage	System action	Control / evidence
Review	Apply approval or staging workflow	Reviewer and decision state
Participate	Register students and capture attendance	Eligibility, event and attendance mode
Evaluate	Collect questionnaire responses where enabled	Response ownership and privacy
Credit	Record eligible participation points	Internal-student rule and audit trail
Close	Prepare report and certificate records	Human review and approved template

3.6 Data Design

The repository contains migrations for student records, programme participation, attendance, surveys, discipline, movement, documents and related operations. The following conceptual groupings help explain the design; they are not a substitute for an entity-relationship diagram generated from the current schema. The final author should add a verified ERD and field definitions from the deployed database.

Table 3.2 Conceptual data groups

Data group	Representative records	Relationship / sensitivity
Identity	User accounts, role data, student identity and profiles	Identity fields are sensitive; access depends on role and ownership
Scholarship and welfare	Scholarship/status forms, B40 and welfare records	Linked to student and reviewed by authorized staff
Discipline	Offenses, evidence, fine applications, stickers	High-sensitivity case records; restricted access
Movement	Movement requests, checkpoint and scan records	Student-linked location or activity history
Programmes	Programs, owners, reviewers and paperwork	Ownership and review state control actions
Participation	Registrations, attendance, survey answers and points	Linked to student/program; eligibility rules apply
Documents and certificates	Private uploads, certificate templates and generated items	Private download authorization and retention needed
Operations	Settings, feature flags, audit and notification records	Administrative data; change tracking required

3.7 Interface and Physical Design

The interface is organized around role-specific navigation, dashboards and task pages. Student pages emphasize self-service, while administrative pages expose lists, filters and review actions within a permitted scope. The project includes light and dark

themes, English and Bahasa Melayu catalogues, responsive layouts and PWA support. These features are present in source documentation; their usability and accessibility should still be validated on institution-approved devices.

The physical deployment should separate browser code from trusted server configuration. The database should not be reachable directly by a mobile or Flutter client. A future Flutter app should authenticate through a documented API with server-side authorization, validation, rate limits and token/session lifecycle controls. Direct database sharing would bypass the Laravel authorization and expose credentials, so it is not a safe extension of the current design.

3.8 Security Architecture and Trust Boundaries

The browser is an untrusted client. It can alter form values, replay requests and access routes that are not linked from the visible menu. Laravel must therefore repeat authorization and validation on every request. The database should accept connections only from application hosts, and application secrets should remain outside source control. Private uploads should use controlled storage paths and should not be made public by a guessable URL.

Trust boundaries also exist between the application and queued workers, mail/push providers, AI services and report-generation libraries. A queued job should carry only the identifiers and minimum data needed to perform its task, then re-check that the record remains eligible before issuing a certificate or notification. Provider calls should use timeouts and bounded retries. AI input should be minimized and reviewed for unnecessary personal data. Generated files should be retained only according to an approved policy.

Security logging should support investigation without becoming a second store of sensitive records. Record administrative actions and key state changes using stable identifiers and actor context, but avoid passwords, authentication tokens, full document contents and unredacted AI prompts. Access to logs should itself be restricted. The final deployment should document who reviews alerts and how suspected exposure is reported.

3.9 Error Handling and Recovery Design

A user-facing operation should have a clear outcome: accepted, rejected with a correctable reason, waiting for background work, or failed and requiring support. When a database transaction updates related records, it should commit as one logical change or roll

back. When a job fails after the main record is saved, the user should see a separate job status rather than a false success. Idempotency or duplicate detection is important for QR scans, payment callbacks and retried provider requests.

Recovery design should account for partial failure. If a certificate batch contains several records and one fails, the system should identify the affected records and allow a safe retry without duplicating completed certificates. If push or email delivery is unavailable, the underlying application status should remain authoritative. If the camera is unavailable at a checkpoint, a supervised manual procedure should preserve an audit trail and later reconciliation.

3.10 Design Decisions and Alternatives

A server-rendered Laravel application centralizes authorization and business rules and reduces the need to duplicate them across clients. A separate Flutter interface can improve device-specific interactions, but it introduces API versioning, authentication lifecycle, offline behavior and mobile release maintenance. Those costs should be compared against the value of an installed app before implementation begins.

QR codes provide a low-cost way to reference an event or checkpoint. They do not by themselves prove that the correct person is present. The system must pair a scan with an authenticated user or supervised identity check, prevent replay where needed, and clearly define any location rule. Biometric methods may have different privacy and device costs; the institution should select attendance controls based on policy, risk and accessibility rather than treating a particular technology as inherently accurate.

A relational database is suitable for records with defined relationships and transactional updates. MySQL/MariaDB is the declared operational engine. SQLite makes the automated suite fast and self-contained, but engine differences require staging tests for migrations, constraints, locking, collation and query behavior. The test and production engines should not be treated as interchangeable proof.

4.0 TEST DESCRIPTION AND RESULTS

4.1 Test Strategy

Verification has three distinct layers. Automated unit and feature tests check code behavior in the configured test environment. Integration and build checks validate application wiring and generated frontend assets. User-acceptance testing asks representative students and staff to complete real tasks and confirm that the system meets institutional needs. Only the first two layers have current evidence in this report.

Testing should cover successful actions, invalid inputs, permission boundaries, failure paths and recovery. For role-scoped pages, tests must exercise direct HTTP requests as well as navigation. For asynchronous jobs and external providers, tests should verify retry, timeout and failure handling without requiring a production delivery claim.

4.2 Verification Environment

Table 4.1 Current validation environment

Item	Observed configuration	Limit
Framework	Laravel Framework 13.24.0	Local repository runtime
PHP CLI	PHP 8.4.0	Does not define the production PHP runtime
Automated database	SQLite :memory: in phpunit.xml	Does not execute MySQL production migrations
Test command	php artisan test --compact	Current local full-suite run
Frontend	Vite 8.2.1 production build	Build success does not test browser behavior
Routes	275 registered routes at verification time	Route registration does not prove each workflow works
Production services	Not exercised	Mail, push, AI, queue production and backup restore remain unverified

4.3 Automated Results

The full Laravel test suite completed successfully on 26 September 2026 with 248 tests and 1,382 assertions. The Vite production build also completed successfully. A current route-list command registered 275 routes. These results supersede the older repository status counts and should be updated after any substantial code change.

The test configuration explicitly uses SQLite in memory. Therefore the pass establishes behavior exercised by the suite under that database engine, not successful

migration or operation against the production MySQL/MariaDB database. The successful Vite build verifies compilation and asset generation, not visual correctness in every browser or role. No end-user UAT sample, sign-off, or production-provider delivery record was supplied.

Table 4.2 Automated validation results

Check	Result	Evidence boundary
Laravel full test suite	PASS: 248 tests, 1,382 assertions	Configured automated tests; SQLite in-memory
Vite production build	PASS: Vite 8.2.1	Frontend compilation and asset manifest
Laravel route registration	PASS: 275 routes listed	Registration only; no per-route runtime guarantee
Production MySQL migration	NOT VERIFIED	Need approved backup, staging DB and migration review
Mail / push / AI delivery	NOT VERIFIED	Need safe test accounts and provider delivery evidence
User acceptance testing	NOT RUN / NO RESULTS PROVIDED	Requires representative students and staff
Backup restoration	NOT VERIFIED	Requires separate restore exercise

4.4 Functional Test Plan

The cases below are proposed acceptance checks derived from the documented functions. A “planned” case is not a result. Record test data, user role, build identifier, actual outcome, defect reference and tester approval when executing them. Avoid using real student personal data unless the institution has approved the test environment and data handling.

Table 4.3 Planned functional test cases

Case	Req.	Action	Expected result	Status
TC-01	FR-01	Sign in with valid and invalid credentials	Valid session; invalid credentials rejected	Planned
TC-02	FR-02	Request an admin URL as a student	Forbidden or safe redirect; no protected data	Planned
TC-03	FR-04	Submit incomplete then completed student profile	Validation is clear; completion state updates	Planned
TC-04	FR-05/06	Submit scholarship-status workflow	Authorized staff can review; student sees permitted status	Planned
TC-05	FR-07	Submit discipline application and attach evidence	Correct case linkage and scoped evidence access	Planned
TC-06	FR-09/10	Use allowed campus movement checkpoint	Valid scan updates correct student movement record	Planned

Case	Req.	Action	Expected result	Status
TC-07	FR-11/12	Register and record programme attendance	Eligible participant and attendance event are linked	Planned
TC-08	FR-13	Run geofence workflow enabled and disabled	Configured rule is enforced; disabled mode behaves as specified	Planned
TC-09	FR-14	Award programme points to eligible and ineligible records	Eligibility rule applied and result traceable	Planned
TC-10	FR-15	Generate and review programme report	Content and page rendering match approved source/template	Planned
TC-11	FR-16	Queue certificate generation and simulate failure	Job status, retry and error handling are accurate	Planned
TC-12	FR-17	Borrow and return a laptop	Inventory and loan state remain consistent	Planned
TC-13	FR-18	Attempt private document access across accounts	Owner and authorized role can access; others denied	Planned
TC-14	FR-20	Revoke an active session	Revoked session cannot perform a protected action	Planned
TC-15	FR-22	Run AI helper with provider unavailable	User receives a safe failure; no sensitive prompt leakage	Planned
TC-16	FR-23	Change a feature setting as authorized and unauthorized user	Allowed change persists; unauthorized change is denied	Planned

4.5 Integration and Operational Testing

Before release, test database migrations against a recent, access-controlled staging copy and a clean database. Capture the current schema and row counts, verify the backup, review each migration for locking and data-conversion risks, apply it in staging, and check rollback or forward-recovery options. The two September 2026 student-column migrations require explicit confirmation against the deployed schema before relying on those fields.

Test private file storage, mail, push and AI integrations using dedicated non-production accounts and non-sensitive data. Confirm timeouts, invalid credentials, provider throttling and retry behavior. For queue-backed certificate jobs, confirm that a worker restarts after failure and that users see the correct final state. Execute a database and file restore into an isolated environment and document the achieved recovery time; do not infer recovery readiness from the existence of backup configuration.

4.6 User Acceptance Testing

UAT should involve representative students and each staff role that owns an in-scope workflow. The institution should approve scenarios, test data, devices and acceptance

thresholds. Each tester should record the task, expected behavior, actual behavior, result, notes and sign-off. Defects should be prioritized and retested after correction.

Table 4.4 User-acceptance test plan

User group	Scenario	Evidence to capture	Status
Student	Profile, document, scholarship, discipline and programme self-service	Device/browser, completion, confusion points, consented feedback	Not run
Lecturer / staff	Programme management, attendance and reporting	Assigned scope, correct records and error recovery	Not run
Scholarship staff	Status and welfare review	Decision trail, privacy and correct student linkage	Not run
Discipline staff	Offense, evidence and application review	Permission checks, case completeness and audit evidence	Not run
Head of Student Affairs	Oversight and approval workflow	Decision visibility, export correctness and accountability	Not run
System administrator	Account, settings, monitoring and maintenance	Privileged-action safeguards and operational clarity	Not run
Guard	Movement lookup and checkpoint scan	Scan reliability, wrong-student protection and device handling	Not run

No UAT outcomes are reported because no completed UAT records were present in the reviewed project materials. These rows are a test plan, not fabricated results.

4.7 Defect Recording and Exit Criteria

Each discovered defect should be reproducible. Record a short title, affected release or commit, role, preconditions, steps, expected result, actual result, severity, supporting screenshot or log with sensitive content redacted, and the retest result. A defect is not closed merely because a developer changed code; a tester should repeat the original failure path and confirm that related permissions and workflows still behave correctly.

Severity should reflect impact and likelihood. A privilege escalation, exposure of private documents, corruption of student records or incorrect official certificate warrants immediate blocking review. A visual spacing issue may be lower priority unless it prevents task completion or creates a dangerous misunderstanding. Production deployment should stop if high-impact security, data-integrity or migration defects remain unresolved.

A release candidate should have a recorded source revision, successful automated suite, successful frontend build, reviewed dependency status, migration rehearsal, approved UAT results, confirmed external service behavior, successful backup-restore

evidence and a rollback or forward-recovery plan. The institution should define acceptable residual defects and sign off the release explicitly.

4.8 Test Data and Coverage Boundaries

Use synthetic records for automated tests and staging wherever possible. A synthetic dataset should include different roles, ownership relationships, missing optional values, boundary dates, invalid state transitions, duplicate requests, large but allowed uploads and provider failures. If a masked production sample is essential, the data owner must approve its use and specify access, retention and deletion controls.

A suite count is not a coverage percentage. The 248 passing test cases demonstrate that the configured tests completed, but they do not show that every controller branch, route, database engine or role combination was exercised. The team should generate and review code-coverage output if a coverage claim is needed, then map untested high-risk paths to planned tests. A passing assertion can also encode an obsolete expectation, so important acceptance criteria need review against current policy.

Test automation should focus on deterministic business rules and security boundaries. UI automation should cover only stable, high-value journeys and should not replace manual accessibility or visual review. For camera-dependent QR workflows, physical device tests should include permission denied, camera unavailable, poor lighting, network interruption and duplicate scan behavior.

5.0 MAJOR FINDINGS AND DISCUSSION

5.1 Findings from Repository Review

The repository documents a broad, multi-role student-affairs platform. Its main strength is the presence of connected workflows across student self-service and administrative operations. Centralized records can make it easier to retrieve a decision and the relevant programme or student context. Whether this reduces processing time or errors at Politeknik Besut remains a question for measured UAT and operational data.

The application uses server-side Laravel authorization and scoped workflows, while its interface uses Blade and compiled assets. This is a coherent basis for a web system because data access and business rules remain on the server. Any future Flutter application should preserve this boundary by calling authenticated APIs rather than connecting to the database directly.

The current automated test suite and frontend build both pass. These checks give useful regression evidence but do not measure user satisfaction, production readiness, data migration safety or the correctness of every live integration. The test database is SQLite in memory, and no MySQL production migration run is included in the evidence.

The project status document dated 14 August 2026 identifies report-template page rendering and official certificate artwork as items requiring confirmation. Those are documented historical review items, not fresh defects confirmed in this validation. They should remain release checks until the current templates receive visual review and institutional approval.

5.2 Advantages

- Role-scoped modules organize work around the responsibilities of students, staff, scholarship administrators, discipline administrators, HEP leadership, system administrators and guards.
- Student-facing functions reduce the need to route every permitted self-service action through a staff member.
- Programme workflows connect registration, attendance, questionnaires, points, reports and certificate records.
- Private documents and server-side provider integrations can be governed at the application boundary.

- Automated regression tests, localized interface catalogues and a production asset build provide a useful base for continued quality work.

5.3 Disadvantages and Limitations

- The system spans many sensitive workflows, so access-control errors can have greater impact than ordinary interface defects.
- Large legacy route closures are harder to isolate and maintain than focused controllers and services, as noted in the repository status documentation.
- External email, push and AI features depend on provider configuration, quotas, availability and privacy terms.
- Official programme-report pages and certificate artwork require template-level institutional review; generated files should not be assumed submission-ready.
- The current test run uses SQLite rather than the production MySQL/MariaDB engine, leaving engine-specific behavior unverified.
- No local UAT evidence establishes task completion, accessibility, user satisfaction or measured operational improvement.

5.4 Entrepreneurial and Practical Value

The practical value is institutional coordination: student-affairs staff can work with connected records and students can access permitted services through one portal. A product or commercialization decision would require evidence beyond the repository. The institution would need to assess licensing and hosting costs, support capacity, security obligations, integration needs, training, accessibility and procurement requirements. This report does not claim a market size, revenue estimate or commercial readiness.

The system could be packaged for similar institutions only after policy and data models are configurable enough to support differences in roles, forms, academic calendars, approval chains and reporting templates. A deployment strategy should separate institution-specific branding and templates from stable workflow logic. Any wider rollout would need a security review and a documented migration path for each customer environment.

5.5 Risk Analysis

Table 5.1 Risks and controls

Risk	Potential effect	Recommended control	Priority
Role or ownership check omitted	Unauthorized disclosure or modification	Permission tests for direct routes and object IDs; periodic access review	High

Risk	Potential effect	Recommended control	Priority
Production migration mismatch	Failed release or missing fields	Backup, staging rehearsal, schema check and approved rollout plan	High
Sensitive data sent to provider	Privacy exposure	Minimize payload, review terms, redact data and log provider use	High
Generated official document is inaccurate	Institutional report or certificate rejected	Golden template, visual QA and human approval before issue	High
Queue or provider outage	Delayed certificate or notification	Retry policy, monitoring, user-visible state and manual fallback	Medium
Mobile QR or camera incompatibility	Attendance or checkpoint interruption	Test target devices and provide supervised manual fallback	Medium
Unrestored backup	Extended outage or unrecoverable records	Schedule and perform isolated restore exercises	High

5.6 Measures for Local Evaluation

The project's intended benefits should be evaluated with measures that the institution can collect consistently. Before a pilot, staff should agree on the task definition, data source, observation period, and calculation rule. The baseline and follow-up should use comparable programme types, user groups and calendar periods. Any change in staffing, policy or activity volume should be noted because it can affect the comparison.

Table 5.2 Candidate measures for a controlled pilot

Measure	Possible calculation	Interpretation limit
Task completion	Completed assigned tasks divided by attempted tasks	Does not explain why users failed; pair with observation
Processing duration	Elapsed time from complete submission to recorded decision	Compare like-for-like cases and exclude documented pauses consistently
Correction rate	Records requiring correction divided by reviewed records	Define what counts as a correction before collecting data
Duplicate entry	Repeated entry events found in sampled workflows	A lower count does not by itself show better decision quality
Access-control outcome	Authorized and unauthorized test requests handled as expected	A finite test set cannot prove absence of all access defects
User-reported clarity	Responses to the same approved question after task completion	Report sample, response rate and question wording
Service reliability	Successful monitored operations divided by eligible operations	Define monitoring window and planned-maintenance treatment

The pilot should identify its participants, recruitment method and sample size before it begins. If participation is voluntary, the team should explain how feedback will be used

and avoid collecting names unless follow-up is required and approved. For each reported percentage, include the numerator and denominator. For each average duration, report the sample size and a distribution or range so that outliers are visible.

Measurements should not encourage staff to process cases quickly at the expense of accuracy or procedural fairness. Quality review can sample decisions and verify required evidence, correct student linkage and appropriate notice. If the process changes during the pilot, record the change and either stratify the results or restart the comparison period. The result should describe association and observed behavior; a short operational comparison does not establish that the software alone caused an outcome.

No local baseline or follow-up values are available in the repository, so this section gives a measurement plan rather than a performance result. After institutional approval and collection, the project team can add a results table with date range, sample, calculation, finding and limitations. Personal student information should be excluded from the report unless publication is authorized and the detail is necessary.

5.7 Discussion Against Related Work

The student-affairs requirements study by Tam et al. highlights the value of eliciting needs and organizing them in a traceable backlog. The MyHEP repository already describes multiple roles and feature areas, but a report derived primarily from source code cannot show whether end users helped prioritize them. The project team should attach its requirements-elicitation evidence and map each accepted requirement to tests and a named policy owner.

The attendance review by Aldabagh and the location-linked QR study by Abdellatif et al. make clear why attendance must be considered as both a technical and human workflow. A QR scan is a capture mechanism, not independent proof of attendance. For MyHEP, evaluation should include duplicate and proxy attempts, device availability, participant understanding, privacy notices and the staff procedure for resolving disputed records. Published studies provide context, not a local success claim.

This comparison also shows a gap in the available project evidence: the code can be tested automatically, but no local participants or operational metrics are reported here. The next useful evaluation should measure task completion, errors, time and user-reported clarity for selected high-volume workflows using the same protocol before and after

deployment. The institution should publish aggregate findings only when the sample and method support them.

5.8 Operational Sustainability

Long-term operation requires named owners for application releases, database changes, account and role reviews, templates, provider credentials, queues, backups and support tickets. The project's module breadth can create maintenance load when a policy changes. A short change record should identify affected roles, forms, migrations, translations, reports and tests before release.

The application should have a support process that distinguishes user assistance from security incidents. A routine profile correction can follow a service workflow; suspected unauthorized access needs restricted escalation and evidence preservation. Staff training should cover how to verify student identity, handle sensitive attachments and explain statuses without sharing another user's data.

Sustainability should be evaluated through operating cost, incident volume, unresolved support needs, dependency maintenance, backup success and user outcomes. Those figures are absent from the repository evidence and should be collected after deployment. If future institutional reuse is considered, the team should also budget for onboarding, migration, configuration, accessibility remediation and ongoing security updates.

6.0 RECOMMENDATION AND CONCLUSION

6.1 Recommendations

1. Complete the official report metadata: approved title, author names and IDs, supervisor, programme, academic session and submission date.
2. Confirm the actual project methodology and replace the draft schedule with dated evidence from the approved plan.
3. Run role-based UAT with representative students and staff, then record results, defects, retests and sign-off. Do not present the proposed UAT matrix as completed testing.
4. Back up the target database and rehearse pending migrations against a production-like MySQL/MariaDB staging database. Confirm the deployed schema before activating any feature that relies on newer student columns.
5. Perform visual review of generated programme reports and certificate templates using the official approved originals. Resolve blank or shifted pages before submission or issuance.
6. Run a security review of role boundaries, private downloads, session revocation, dependency advisories, provider payloads, secrets, logging and data-retention rules.
7. Test mail, push, AI and queue-backed workflows in a non-production environment, including timeouts, throttling and worker recovery.
8. Execute an isolated backup restoration and document recovery objectives, actual duration, data verification and accountable owner.
9. For a future Flutter client, define versioned authenticated APIs and authorization tests. Keep the database private to the backend.
10. Refresh this report's test counts and screenshots after the final code change, and update all page numbers after institutional editing.

6.2 Recommended Evaluation Sequence

The project team can complete the remaining evaluation in an evidence-first order. First, freeze a source revision and confirm the official project metadata and workflow policies. Next, review access boundaries and rehearse migrations in staging after taking and verifying a backup. Then exercise external services and background jobs with test credentials. After the technical checks pass, conduct role-specific UAT on the approved browser and device matrix. Correct defects, repeat affected tests and obtain institutional sign-off before making a release or impact claim.

For each stage, retain a compact evidence record: date, environment, version, tester or approver role, test cases, outcome, defect reference and follow-up. The final report can

then replace its placeholders and “not verified” labels with traceable results. If a check was not performed, retain that status rather than inferring success from a neighboring check.

6.3 Conclusion

MyHEP is documented in the repository as a Laravel-based student-affairs platform that joins student self-service with scoped administrative workflows for scholarship and welfare, discipline, movement, programmes, documents and operations. The current local validation provides concrete but bounded evidence: 248 automated tests passed with 1,382 assertions, the Vite production build succeeded, and Laravel registered 275 routes. These results support continued testing and review; they do not alone establish production readiness.

The report identifies the work still required to complete an academic and operational evaluation. The author team must confirm its official project details, methodology and schedule. The institution must conduct and sign off UAT, validate migrations against the intended MySQL/MariaDB deployment, test external services and restore backups, and approve official report and certificate templates. Until that evidence is recorded, claims about user impact, production operation and acceptance should remain open.

REFERENCES

- Abdellatif, H., Sirasanagandlae, S. R., Al-Mushaiqri, M., & Sakr, H. F. (2022). Location-linked QR code as a safe tool for recording classroom attendance during COVID-19 pandemic: Perspectives of medical students. *Medical Science Educator*, 32(5), 971–974. <https://doi.org/10.1007/s40670-022-01625-7>
- Aldabagh, M. (2024). A review of students attendance management systems. *Eurasian Journal of Science and Engineering*, 10(3), 136–145. <https://doi.org/10.23918/eajse.v10i3p13>
- Tam, N. R. A., Prasetyo, K. W., & Kristanto, B. K. (2022). A student affairs management information system requirements at STIKI Malang using Agile requirements engineering. *J-INTECH*, 10(1), 21–29. <https://doi.org/10.32664/j-intech.v10i1.675>
- Politeknik Besut. (2026). Guideline and content of technical report [Course/project guideline].
- Politeknik Besut. (n.d.). Technical report general format [Formatting guideline].
- StudentEdge project team. (2026). System overview, project status, role access matrix, workflows, developer guide and testing/UAT documentation [Software repository documentation].

APPENDIX A REQUIREMENTS TRACEABILITY

Table A.1 Requirements traceability

Requirement	Feature	Verification evidence still required
FR-01	Authentication and recovery	Automated auth tests; role-specific UAT
FR-02	Role and scope checks	Authorization tests for direct requests and data ownership
FR-04	Student profile	Profile feature tests; student UAT
FR-05/06	Scholarship and welfare	Workflow tests; scholarship staff UAT
FR-07/08	Discipline and stickers	Case/permission tests; discipline staff UAT
FR-09/10	Movement and QR	Checkpoint tests on approved devices; guard UAT
FR-11/12/13	Programme, attendance, survey, geofence	Integration tests and event-owner/student UAT
FR-14	Participation points	Eligibility tests against policy examples
FR-15	Programme reporting	Golden-file visual review and content reconciliation
FR-16	Certificates	Queue failure tests and approved artwork review
FR-17	Laptop lending	Inventory/loan state transition tests
FR-18	Private documents	Cross-account direct-download authorization tests
FR-20	Sessions and notification	Session revoke tests and safe provider checks
FR-22	AI integration	Provider unavailable, privacy and timeout tests
FR-23/24	Administration and locale	Privileged action tests; English/BM copy review

APPENDIX B AUTHOR COMPLETION CHECKLIST

Table B.1 Author completion checklist

Item	Required action	Complete
Project identity	Confirm official title, author(s), IDs, programme, supervisor and session	[]
Declaration	Review originality statement and sign only when accurate	[]
Method	Describe the actual method and attach supporting project evidence	[]
Schedule	Replace all date placeholders with approved dates and actual status	[]
Literature	Verify full bibliographic metadata and add any required institutional sources	[]
System design	Insert verified schema/ERD and institution-approved interface figures if required	[]
Test evidence	Attach fresh test output and identify build/commit tested	[]
UAT	Conduct and document acceptance tasks with representative users	[]
Deployment	Validate MySQL migrations, integrations, security and backup restoration	[]
Templates	Obtain institutional approval for reports, certificates and signatures	[]
Pagination	Update contents and list-of-tables/figures page numbers after edits	[]
Submission	Confirm final page count, binding, signatures and department requirements	[]